

PROJECT REPORT

On

Credit Risk Score Analysis

Submitted in partial fulfilment of the requirement for the Course
Cloud Computing & Devops in Financial Services(23CS030)
of

**COMPUTER SCIENCE AND ENGINEERING
B.E. Batch-2023**

in

MAY-2026



Submitted By

**Under the Guidance of
Saubhagya Verma**

Sanya Aggarwal(2310991123)

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY
PUNJAB**

CERTIFICATE

This is to be certified that the project entitled “Credit Risk Score Analytics” has been submitted for the Bachelor of Computer Science Engineering at Chitkara University, Punjab during the academic semester January 2026- May-2026 is a bonafide piece of project work carried out by “Sanya Aggarwal (2310991123) of the group G33” towards the partial fulfillment for the award of the course Integrated Project (22CS038) under the guidance of “Saubhagya Verma” and supervision.

Sign of the Project Guide

Saubhagya Verma
(Trainer)

CANDIDATE'S DECLARATION

We, **SANYA AGGARWAL (2310991123)** of the group **G33**, B.E.-2023 of the Chitkara University, Punjab hereby declare that the Integrated Project (22CS038) Project Report entitled “**Credit Risk Score Analytics**” is an original work and data provided in the study is authentic to the best of our knowledge. This report has not been submitted to any other Institute for the award of any other course.

Signature of the student:

Name : Sanya Aggarwal

ID No. : 2310991123

Place : Chitkara University

Date : May 28, 2026

ACKNOWLEDGEMENT

It is our pleasure to be indebted to various people, who directly or indirectly contributed in the development of this work and who influenced my thinking, behavior and acts during the course of study.

We express our sincere gratitude to all for providing me an opportunity to undergo Integrated Project as the part of the curriculum.

We are thankful to “Saubhagya Sir” for his support, cooperation, and motivation provided to us during the training for constant inspiration, presence and blessings.

We also extend our sincere appreciation to “**Dr. Gaganpreet Kaur**” who provided his valuable suggestions and precious time in accomplishing our Integrated project report.

Lastly, We would like to thank the almighty and our parents for their moral support and friends with whom we shared our day-to day experience and received lots of suggestions that improve our quality of work.

Name : Sanya Aggarwal
ID No. : 2310991123

ABSTRACT

The rapid growth of the financial technology (FinTech) sector has significantly increased the need for automated, scalable, and reliable financial decision-making systems. Financial institutions process large volumes of customer loan applications daily, making credit risk assessment a crucial part of lending operations. Traditional credit evaluation methods generally rely on manual verification, paperwork, and human judgment, which often lead to delays, inconsistency, and increased chances of human error. These limitations highlight the need for intelligent systems capable of automating financial risk analysis efficiently and accurately. The Credit Risk Analytics project is developed to provide an automated platform for evaluating customer creditworthiness using predefined financial parameters and business logic. A major component of the system is the calculation of the Debt-to-Income (DTI) ratio, which is used to evaluate a customer's repayment capability and overall financial stability. Based on the calculated DTI value and customer credit score, the system automatically categorizes applications into approved or rejected categories. The application implements predefined business rules for financial risk analysis. Loan applications are rejected when the customer credit score is below the required threshold or when the DTI ratio exceeds the acceptable limit. Eligible applicants are approved and assigned dynamic interest rates according to their financial risk profile. Pandas is used for efficient financial data processing and transformation, while PostgreSQL is used as the backend database for storing processed customer records and historical analytics information. The platform also includes features for database logging and retrieval of historical financial records. From a Cloud Computing and DevOps perspective, the application demonstrates practical implementation of modern deployment methodologies. The system is containerized using Docker, where separate containers are used for the application and database services, enabling modular architecture, and simplified deployment. Docker networking facilitates communication between application and database containers while maintaining operational flexibility. For cloud deployment, the project is hosted on an Amazon Web Services (AWS) EC2 ubuntu instance, making the platform publicly accessible over the internet.

KEYWORDS: Credit Risk Analytics, FinTech, Cloud Computing, DevOps, Docker, AWS EC2, PostgreSQL, Streamlit, Financial Analytics, Containerization.

TABLE OF CONTENTS

Certificate	iii
Candidate's Declaration	iv
Acknowledgement	v
Abstract	vii
Table of Contents	viii
List of Figures	ix
List of Tables	x
Executive Summary	xii
Chapter 1: Introduction to the Project	1
Chapter 2: Objectives and Requirement Specification	5
Chapter 3: Methods and Design Approach	10
Chapter 4: System Architecture	14
Chapter 5: Programming and Working Environment	18
Chapter 6: Database Analysis, Design and Implementation	20
Chapter 7: Program Structure and GUI Construction	22
Chapter 8: Code Implementation and Database Connections	25
Chapter 9: System Testing	26
Chapter 10: Limitations and Future Scope	27
Chapter 11: Conclusion	30
Code Snippets and Implementation Files	31
Testing Outputs and Deployment Proof	34
References	35
Appendix	36

LIST OF FIGURES

Figure No.	Title
Figure (1)	Credit Risk Score Analytics System Architecture
Figure (2)	Authentication and Transaction Data Flow
Figure (3)	App.py for Credit Risk Score Analytics
Figure (4)	Main Body Pipeline
Figure (5)	Ubuntu Docker Setup
Figure (6)	WebPage
Figure(7)	Audit Records

LIST OF TABLES

Table No.	Title
Table (1)	Software Requirements
Table (2)	Hardware Requirements
Table (3)	Technologies Used in Credit Risk Score Analytics
Table (4)	Main Database Entities
Table (5)	Testing Summary

1. INTRODUCTION TO THE PROJECT

1.1 Background

The financial sector has undergone significant transformation due to rapid technological advancements and the emergence of Financial Technology (FinTech) systems. Traditional banking and financial institutions increasingly depend on digital platforms for processing customer information, evaluating loan applications, and managing financial services efficiently. Among these operations, credit risk assessment plays a crucial role because financial institutions must determine whether a customer is financially capable of repaying loans or credit obligations. Traditionally, credit assessment processes relied heavily on manual verification, paperwork, customer interviews, and human judgment. Automated credit risk analysis helps organizations process large volumes of customer data quickly while ensuring consistent and accurate evaluation. Such systems analyze multiple financial indicators and predefined business rules to determine customer eligibility for financial services. This approach minimizes risks associated with poor lending decisions and improves overall operational efficiency. At the same time, the adoption of cloud computing and DevOps practices has significantly transformed how financial applications are developed, deployed, and managed. Cloud platforms enable scalable infrastructure, remote accessibility, and improved system reliability. Similarly, DevOps methodologies support efficient deployment, automation, containerization, and service management, making software systems easier to maintain and scale. Considering these technological advancements and financial industry requirements, the Credit Risk Analytics project is developed to provide an automated, cloud-based, and containerized solution for credit risk assessment. The application is deployed using Docker containers on an AWS EC2 cloud environment, demonstrating practical implementation of Cloud Computing and DevOps concepts in financial systems. The project not only improves efficiency and automation in credit assessment but also provides practical exposure to financial analytics, database integration, cloud deployment, Linux administration, Docker containerization, and real-world DevOps implementation.

1.2 Problem Statement

In the modern financial ecosystem, banks and financial institutions receive a large number of loan applications and credit requests on a daily basis. Evaluating customer financial stability and repayment capability is one of the most critical tasks in the lending process. Traditional methods of credit assessment often involve manual verification of financial documents, human judgment, and time-consuming approval procedures. These approaches may result in delays, inconsistency, operational inefficiencies, and an increased probability of human errors. Financial institutions face multiple challenges while evaluating creditworthiness manually. Analysts are required to examine customer income, debt obligations, repayment history, and financial behavior before making lending decisions. As the number of applications increases, managing and analyzing large volumes of financial data becomes increasingly difficult. Manual processes also make it challenging to maintain consistency in decision-making, as approvals may vary depending on individual judgment and interpretation. Another major limitation of traditional systems is the lack of automation and real-time analytics. Many existing systems do not provide intelligent mechanisms for evaluating customer risk using predefined financial parameters. As a result, financial institutions may experience delays in approval, higher chances of poor lending decisions, increased financial risk, and reduced operational efficiency. Additionally, many traditional financial systems lack cloud-based accessibility and scalable deployment mechanisms. Applications hosted on local systems often face limitations related to accessibility, maintenance, scalability, and infrastructure management. Without proper deployment strategies, maintaining secure and reliable financial systems becomes difficult. The absence of modern DevOps practices further creates challenges in application deployment, dependency management, scalability, and service reliability. Traditional software deployment methods often require manual configuration, making systems difficult to maintain and prone to deployment-related failures. Therefore, there is a strong need for an automated, scalable, and cloud-deployed credit risk assessment system that can efficiently process customer financial information, minimize manual effort, improve consistency in decision-making, and provide reliable risk analysis. To address these limitations, the Credit Risk Analytics project is developed to automate customer risk assessment using predefined business logic, database integration, Docker containerization, and AWS cloud deployment.

1.3 Purpose and Overview of the Report

The main purpose of the Credit Risk Analytics project is to develop an automated and efficient platform for evaluating customer creditworthiness using predefined financial parameters and business rules. The system is designed to reduce manual effort involved in traditional credit assessment processes and provide faster, more consistent, and reliable lending decisions. The project focuses on analyzing customer financial information such as income, monthly debt obligations, and credit score to determine the financial risk associated with loan approval. By calculating the Debt-to-Income (DTI) ratio and applying rule-based analytics, the system automatically categorizes customers into approved or rejected loan categories and generates suitable interest rates for eligible applicants. Another important purpose of the project is to demonstrate the practical implementation of Cloud Computing and DevOps concepts in financial systems. The application is containerized using Docker and deployed on an AWS EC2 cloud environment, allowing scalable, portable, and publicly accessible deployment. The project also aims to provide hands-on experience with Linux server management, Docker containerization, cloud deployment, database integration, and modern software deployment practices. The project ultimately aims to improve operational efficiency, minimize human errors, automate financial analytics, and provide a centralized platform for managing and monitoring credit risk evaluation processes.

1.3.1 Overview of the Project

The Credit Risk Analytics project is a cloud-based financial analytics platform developed for automating customer credit risk assessment. The application processes financial datasets, evaluates customer eligibility for loans, and stores historical analytics records using a structured database system. The system accepts customer financial information including annual income, monthly debt, and credit score as input parameters. Based on this information, the application calculates the Debt-to-Income (DTI) ratio, which is one of the most important indicators used for measuring repayment capability and financial stability. Using predefined business logic, the platform determines whether the loan application should be approved or rejected. The project is developed using Python, Pandas, and Streamlit. Python is used for implementing the analytics logic and financial calculations, while Pandas handles efficient data processing and transformation operations. Streamlit is

used to create an interactive and user-friendly dashboard interface that allows users to generate sample datasets, run analytics operations, and view processed financial information visually. For data storage and historical record management, PostgreSQL is used as the backend database. The application stores processed customer analytics data and supports retrieval of historical records through the dashboard interface. From a DevOps perspective, the entire system is containerized using Docker. Separate Docker containers are used for the application and database services, enabling isolated execution and easier dependency management. Docker networking is configured to establish communication between containers securely. The application is deployed on an Amazon Web Services (AWS) EC2 Ubuntu instance, making the system accessible over the internet using public IP-based deployment. AWS security groups are configured to allow secure SSH access and application access through specific ports. Overall, the project demonstrates the integration of financial analytics, database systems, cloud deployment, Docker containerization, and DevOps practices into a single scalable platform for automated credit risk assessment. The application stores processed customer analytics data and supports retrieval of historical records through the dashboard interface. From a DevOps perspective, the entire system is containerized using Docker. Separate Docker containers are used for the application and database services, enabling isolated execution and easier dependency management. Docker networking is configured to establish communication between containers securely. The application is deployed on an Amazon Web Services (AWS) EC2 Ubuntu instance, making the system accessible over the internet using public IP-based deployment. AWS security groups are configured to allow secure SSH access and application access through specific ports. Overall, the project demonstrates the integration of financial analytics, database systems, cloud deployment, Docker containerization, and DevOps practices into a single scalable platform for automated credit risk assessment. The system accepts customer financial information including annual income, monthly debt, and credit score as input parameters. Based on this information, the application calculates the Debt-to-Income (DTI) ratio, which is one of the most important indicators used for measuring repayment capability and financial stability. The application processes financial datasets, evaluates customer eligibility for loans, and stores historical analytics records using a structured database system. The system accepts customer financial information including annual income, monthly debt, and credit score as input parameters

2.OBJECTIVES & REQUIREMENT SPECIFICATION

2.1 Project Objective

The primary objective of the Credit Risk Analytics project is to develop an automated and efficient system for evaluating customer creditworthiness using predefined financial parameters and business rules. By automating the evaluation process, the system helps financial institutions make faster, more reliable, and consistent lending decisions while minimizing operational complexity. Another important objective of the project is to analyze customer financial stability using critical indicators such as annual income, monthly debt obligations, and credit score. Based on predefined business logic, the application automatically categorizes loan applications into approved or rejected categories and determines suitable interest rates for eligible customers according to their financial risk level. This helps improve decision accuracy and reduces risks associated with poor lending decisions. The project also focuses on developing an interactive and user-friendly dashboard using Streamlit to simplify financial analytics and improve accessibility. PostgreSQL is integrated to maintain historical customer records and support database logging for future analysis and auditing purposes. From a Cloud Computing and DevOps perspective, another major objective of the project is to demonstrate practical implementation of modern deployment methodologies. The application is containerized using Docker to ensure dependency isolation, portability, and simplified deployment. Separate containers are maintained for the application and database services to improve scalability and maintainability. Furthermore, the project is deployed on Amazon Web Services (AWS) EC2 cloud infrastructure to provide real-world accessibility and practical exposure to cloud deployment, Linux server administration, Docker networking, and service management. Overall, the project aims to integrate financial analytics, cloud computing, database systems, and DevOps practices into a single scalable platform capable of automating credit risk assessment while improving efficiency, consistency, and accessibility in financial decision-making. The project also focuses on developing an interactive and user-friendly dashboard using Streamlit to simplify financial analytics and improve accessibility.

2.2 Software Requirements

The Credit Risk Analytics project is developed using various software technologies, frameworks, libraries, and cloud services to ensure efficient processing, smooth deployment, and reliable system performance. The software requirements include development tools, programming environments, database systems, deployment technologies, and supporting software necessary for successful implementation of the project..

Table (1): Software Requirements

Software/Tool	Purpose
Operating System	Windows/Ubuntu Linux
Programming Language	Python
Frontend Framework	Streamlit
Data Processing Library	Pandas
Database Management System	PostgreSQL
Database Connectivity	Psycopg2
Containerization tool	Docker

The project is primarily developed using Python as the core programming language due to its simplicity, flexibility, and strong support for data processing and analytics operations. Streamlit is used as the frontend framework for building an interactive and user-friendly dashboard that allows users to visualize analytics results and interact with the system efficiently. Pandas is utilized for data manipulation, transformation, and processing of customer financial datasets, while Psycopg2 is used for establishing communication between the application and PostgreSQL database. PostgreSQL is used as the backend database management system for storing processed customer financial records and maintaining historical analytics information. Docker is used for containerizing both the application and database services to simplify deployment, dependency management, and portability across different environments

2.3 Hardware Requirements

The Credit Risk Score Analysis project can be developed and tested on a normal laptop or desktop computer. Since the project uses multiple services such as frontend, backend microservices, PostgreSQL, Redis, and Docker containers, the system should have enough processing power and memory to run all components smoothly. A system with at least an Intel Core i5 processor or equivalent is recommended. The processor should be capable of handling multiple running services at the same time. For better performance, an Intel Core i7 or equivalent processor can be used, especially when running Docker containers and backend services together. The minimum required RAM for running the project is 8 GB. However, 16 GB RAM is recommended for smoother performance because Docker, PostgreSQL, Redis, frontend server, and backend services may run simultaneously. More RAM helps reduce system lag and improves development experience. . This space is required for project files, Docker images, containers, database data, dependencies, node modules, Python packages, and temporary files. SSD storage is preferred over HDD because it provides faster application startup, faster Docker performance, and quicker file access. A stable internet connection is required for installing dependencies, pulling Docker images, pushing code to GitHub, accessing AWS services, and deploying the application on the cloud. Internet access is also required for testing the hosted frontend dashboard and backend endpoints deployed on AWS. A standard keyboard, mouse, and display are sufficient for development and testing. A screen resolution of 1366×768 or higher is recommended so that the dashboard, code editor, terminal, and browser can be used comfortably. For better productivity, a larger screen or external monitor can be used.

Table (2): Hardware Requirements

Component	Minimum Requirement
Processor	Intel i5 / AMD Ryzen 5 or equivalent
RAM	8 GB or higher
Storage	10 GB free disk space
Network	Stable internet connection
Operating System	Windows, Linux, or macOS with Docker support

2.4 Functional Requirements

The functional requirements of the Credit Risk Analytics project define the core operations and functionalities that the system must perform to achieve its intended objectives. The system is designed to automate customer credit risk assessment by accepting financial information, processing customer data, and generating analytical results based on predefined business logic. The platform must be capable of collecting customer financial details such as annual income, monthly debt obligations, and credit score to evaluate overall financial stability and repayment capability. The system should calculate the Debt-to-Income (DTI) ratio automatically using customer financial information and apply predefined financial rules to determine loan eligibility. Based on the calculated DTI ratio and customer credit score, the application must classify loan requests into approved or rejected categories and calculate suitable interest rates for eligible applicants. The platform should also provide an interactive dashboard for users to generate sample datasets, execute analytics operations, and visualize financial results in a user-friendly manner. Additionally, the system must support database integration for storing processed customer records and maintaining historical analytics information. Users should be able to retrieve historical database records for auditing and future analysis purposes. From a deployment perspective, the application should support Docker-based containerization for simplified deployment and maintain secure communication between application and database services. Furthermore, the system should be deployed on a cloud platform such as AWS EC2 to ensure public accessibility, scalability, and real-world availability of the application. The system should calculate the Debt-to-Income (DTI) ratio automatically using customer financial information and apply predefined financial rules to determine loan eligibility. Based on the calculated DTI ratio and customer credit score, the application must classify loan requests into approved or rejected categories and calculate suitable interest rates for eligible applicants. The platform should also provide an interactive dashboard for users to generate sample datasets, execute analytics operations, and visualize financial results in a user-friendly manner. Additionally, the system must support database integration for storing processed customer records and maintaining historical analytics information.

2.5 Non - Functional Requirements

Non-functional requirements describe the quality, performance, security, usability, and reliability expectations of the Credit Risk Score Analytics system. These requirements explain how the system should perform rather than what specific features it should provide. The system shall be secure. Since Credit Risk Score Analytics is related to banking and financial data, security is one of the most important requirements. The system shall protect user accounts, passwords, transaction records, payment details, and consent data from unauthorized access. JWT authentication, password hashing, API key validation, role-based access control, CORS configuration, rate limiting, and security headers shall be used to improve security. The system shall be reliable. The system should work correctly during normal usage and should handle errors without crashing. If one request fails, the complete application should not stop working. Backend services should return proper responses, and the frontend should display meaningful messages to users. The system shall be scalable. Since Credit Risk Score Analytics follows a microservices architecture, each service can be scaled independently in the future. For example, if the Analytics Service receives more requests, it can be scaled without affecting the Auth Service or Banking API Service. This makes the system suitable for future expansion. The system shall be maintainable. The project should be divided into separate modules and services so that changes can be made easily. Auth Service, Banking API Service, Analytics Service, frontend dashboard, database, and cache should have clearly defined responsibilities. This makes the code easier to understand, update, debug, and improve. The system shall provide good performance. API responses should be generated within a reasonable time. Redis caching should be used to improve analytics response speed. The frontend dashboard should load smoothly and display charts, tables, and transaction data without unnecessary delay. The system shall be user-friendly. The frontend dashboard should have a clean and simple interface. Users should be able to understand transaction details, fraud alerts, analytics graphs, and service health status easily. Navigation between pages such as Dashboard, Transactions, Analytics, Risk Monitoring, API Health, and Settings should be simple. The system shall be available during demonstration and testing. When deployed locally using Docker Compose, all required services should start properly. The frontend, backend services, database, and cache should be accessible through their configured ports..

3.METHODS AND DESIGN APPROACH

3.1 Development Methodology

The development methodology used for Credit Risk Score Analytics follows an iterative and modular development approach. Since the project is based on a microservices architecture, the complete system was divided into smaller independent components such as the Auth Service, Banking API Service, Analytics Service, frontend dashboard, database, cache, and deployment setup. Each module was designed, developed, tested, and improved step by step. This approach helped in reducing complexity and made it easier to identify errors, update features, and connect services gradually. In the initial phase, the main requirements of the project were identified. The system needed to support secure user authentication, payment initiation, transaction management, consent handling, fraud risk scoring, analytics visualization, and cloud deployment. After identifying these requirements, the overall architecture of Credit Risk Score Analytics was planned. A microservices-based structure was selected because it allows different services to perform separate responsibilities and makes the system more scalable and maintainable. After planning the architecture, the backend services were developed using FastAPI. The Auth Service was created first because authentication is required before accessing protected banking APIs. It includes user registration, login, JWT token generation, password hashing, API key handling, and role-based access control. After that, the Banking API Service was developed to manage transactions, payments, consent records, and real-time transaction streaming. Then the Analytics Service was implemented to generate insights such as transaction summary, fraud risk distribution, spending by category, and API health information. The frontend dashboard was developed using React. The dashboard was designed to provide a clean and professional fintech-style interface where users can view transaction data, analytics charts, fraud alerts, live transaction activity, and service health status. Different pages such as Dashboard, Transactions, Analytics, Risk Monitoring, API Health, and Settings were created to organize the system features properly. Axios was used to connect the frontend with backend APIs, and chart libraries were used to display data visually.

3.2 Microservices Design Approach

The Microservices Design Approach used in Credit Risk Score Analytics is based on dividing the complete banking analytics platform into small, independent, and manageable services. Instead of creating one large monolithic application where all features are placed together, Credit Risk Score Analytics separates the system into different services, where each service performs a specific responsibility. This approach makes the project easier to develop, test, maintain, scale, and deploy. In Credit Risk Score Analytics, the backend is mainly divided into three important microservices: Auth Service, Banking API Service, and Analytics Service. Each service is developed using FastAPI and works independently while communicating with other services through APIs. This separation helps in keeping the system organized because authentication, banking operations, and analytics processing are handled by different services. The Auth Service is responsible for all authentication and authorization-related tasks. It manages user registration, user login, JWT token generation, password hashing, API key handling, and role-based access control. Since security is a very important part of any banking system, separating authentication into its own service makes the system more secure and easier to manage. Any changes related to login, tokens, or user roles can be made in the Auth Service without affecting other parts of the application. The Banking API Service is responsible for the core banking operations of the system. It handles payment initiation, transaction management, account-related operations, consent handling, and real-time transaction streaming. This service acts as the main banking logic layer of Credit Risk Score Analytics. When a user creates a payment or views transaction history, the request is processed by the Banking API Service. It validates the data, communicates with the PostgreSQL database, and returns the required response to the frontend. The Analytics Service is responsible for processing transaction data and generating useful financial insights. It provides analytics such as transaction summaries, fraud risk distribution, spending by category, risk monitoring, and service health information. This service reads transaction-related data, processes it, and sends the results to the frontend dashboard. Redis caching is also used with the Analytics Service to improve performance by storing frequently requested analytics results temporarily. The React frontend dashboard communicates with these backend microservices using HTTP API requests. For example, login requests are sent to the Auth Service, transaction-related requests are sent to the Banking API Service, and dashboard analytics requests are sent to the Analytics Service.

3.3 Data Flow

The data flow of Credit Risk Score Analytics explains how information moves between different components of the system. The main flow starts from the React frontend dashboard, where the user interacts with the application. When a user performs an action such as login, viewing transactions, creating a payment, or checking analytics, the frontend sends an API request to the appropriate backend service. Each backend service processes the request, communicates with the database or cache if required, and then sends a response back to the frontend. When a user logs into the system, the login request is sent from the React dashboard to the Auth Service. The Auth Service receives the email and password entered by the user. It verifies the user details by checking the stored information in the PostgreSQL database. If the credentials are valid, the Auth Service generates a JWT token and sends it back to the frontend. This token is stored temporarily on the client side and is included in future API requests to access protected services. For banking operations, the frontend sends requests to the Banking API Service. When a user creates a payment, the Banking API Service validates the payment details such as amount, account information, category, and description. After validation, the payment or transaction record is stored in PostgreSQL. When the user requests transaction history, the Banking API Service fetches transaction records from the database and sends them back to the frontend dashboard. The dashboard then displays this data in the form of tables, cards, and transaction feeds. Consent-related data also flows through the Banking API Service. When consent information is created or managed, the service stores the consent record in the database. This ensures that access to financial data is controlled and follows the idea of open banking, where user permission is important before sharing or accessing banking data. The Analytics Service handles the flow of analytical data. It reads transaction-related data from PostgreSQL and processes it to generate useful insights. These insights include total number of transactions, fraud risk distribution, spending by category, transaction summaries, and service health information. To improve performance, Redis is used as a caching layer. When analytics data is requested frequently, Redis can return cached results instead of repeatedly querying the database. This reduces database load and improves response time. Fraud risk scoring is also part of the data flow. When transaction data is processed, the system checks factors such as transaction amount, country, time, and unusual behavior.

3.3 Security Design Approach

The security design approach of Credit Risk Score Analytics focuses on protecting user accounts, banking data, transactions, APIs, and system communication. Since the project is related to financial technology and banking operations, security is one of the most important parts of the system. The project uses multiple security mechanisms such as authentication, authorization, password hashing, JWT tokens, API keys, role-based access control, BOLA protection, rate limiting, CORS configuration, and security headers. The first security layer is user authentication. Credit Risk Score Analytics allows only registered users to log into the system. credentials against the records stored in PostgreSQL. If the details are correct, the system generates a JWT token. This token is then used to access protected APIs. Without a valid token, users cannot access sensitive banking services such as transactions, payments, analytics, or user-specific data. Password protection is another important part of the security design. User passwords are not stored in plain text. Instead, passwords are hashed using bcrypt before being saved in the database. Hashing ensures that even if the database is exposed, the original passwords cannot be easily read. During login, the entered password is compared with the stored hashed password to verify the user. Authorization is handled using role-based access control. Different users may have different roles, such as normal user, third-party provider, or administrator. Each role is allowed to access only specific parts of the system. For example, a normal user may view their own transactions, while an administrator may be allowed to monitor system health or analytics. This prevents unnecessary access to sensitive features. Credit Risk Score Analytics also includes protection against Broken Object Level Authorization, commonly known as BOLA. BOLA occurs when a user can access another user's data by changing an object ID in the API request. To prevent this, the system checks whether the requested resource belongs to the authenticated user before returning the data. This is very important in banking systems because transaction and account data must remain private. API key management is used to secure service-level or third-party access. API keys help identify trusted clients or services before allowing them to access selected endpoints. This adds another layer of protection, especially in an open banking environment where third-party providers may interact with banking APIs. Rate limiting is used to protect the system from excessive requests. If a user or attacker sends too many requests in a short period, rate limiting can restrict access temporarily.

4. SYSTEM ARCHITECTURE

4.1 Architecture Overview

The architecture of Credit Risk Score Analytics is designed as a cloud-native microservices-based system that simulates the working of a modern open banking and fintech analytics platform. The main purpose of this architecture is to divide the complete application into separate layers and services so that each component can perform its own responsibility in a clear and organized manner. This makes the system easier to develop, test, deploy, maintain, and scale. Credit Risk Score Analytics mainly consists of a React frontend dashboard, FastAPI backend microservices, PostgreSQL database, Redis cache, Docker-based local deployment, and AWS-based cloud deployment components. The frontend acts as the user interface of the system, while the backend services handle authentication, banking operations, transaction management, consent handling, fraud risk scoring, analytics processing, and service monitoring. The frontend layer is developed using React. It provides a modern fintech-style dashboard where users can view transaction details, fraud alerts, analytics charts, risk monitoring information, API health status, and system settings.

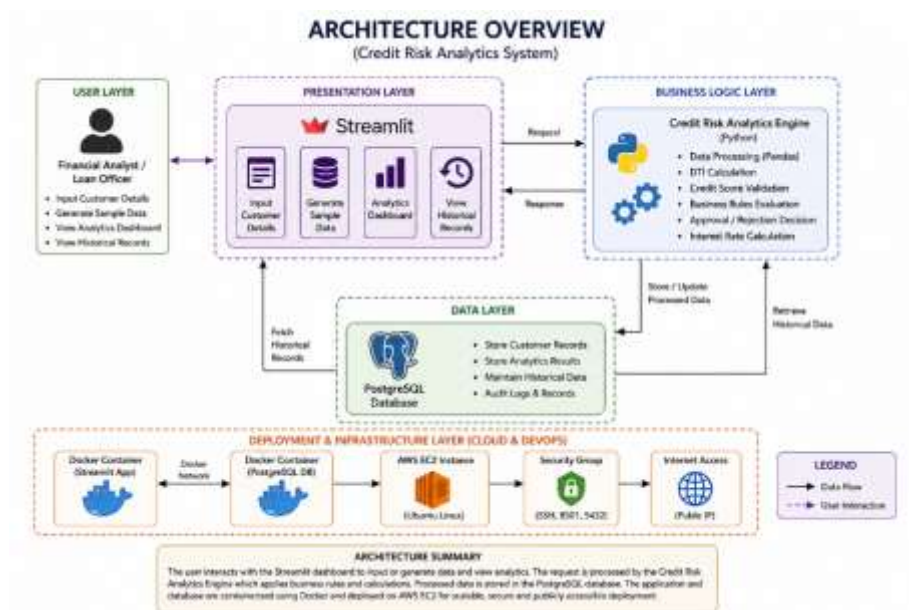


Figure (1): Credit Risk Score Analytics System Architecture

4.2 Component Data Flow

The frontend communicates with backend services using HTTP API requests and WebSocket connections. It sends requests to different backend services depending on the required operation. PostgreSQL stores important records such as users, payments, transactions, account details, and consent information. Redis stores temporary data such as analytics cache and rate-limiting information. In cloud deployment, the data flow is partially moved to AWS. The frontend dashboard is hosted on AWS S3 as a static website. Backend Docker images are stored in Amazon ECR, and the Auth Service is deployed on AWS ECS Fargate. When the hosted frontend sends authentication requests, the deployed Auth Service processes them in the cloud. This proves that Credit Risk Score Analytics can be extended into a complete cloud-based architecture. In the future, the Banking API Service and Analytics Service can also be deployed on ECS Fargate, while PostgreSQL can be moved to AWS RDS and Redis can be moved to AWS ElastiCache. Overall, the component data flow of Credit Risk Score Analytics shows how user actions are passed from the frontend to backend services, how backend services process and store data, how analytics are generated, and how security is maintained during communication. This flow makes the system organized, scalable, and suitable for demonstrating real-world fintech and cloud-native banking applications.

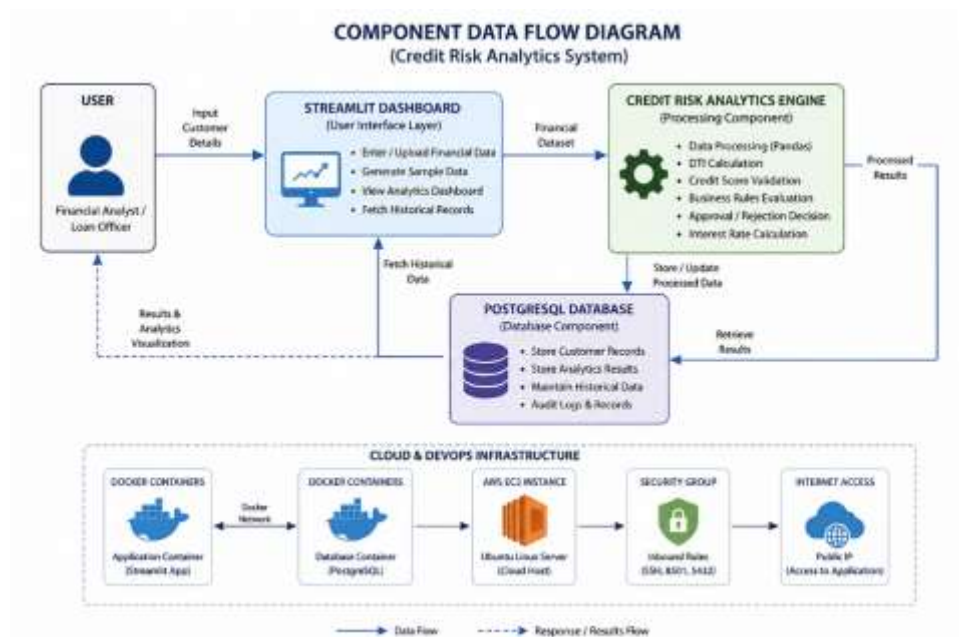


Figure (2): Component Data Flow

4.3 Local Deployment Architecture

The local deployment architecture of Credit Risk Score Analytics is designed to run the complete project on a single laptop or desktop system using Docker and Docker Compose. Although the project runs locally, it follows a microservices-based structure similar to a real-world distributed banking application. This helps in testing, development, debugging, and project demonstration without requiring complete cloud infrastructure. In local deployment, each major component of Credit Risk Score Analytics runs inside a separate Docker container. The frontend dashboard, Auth Service, Banking API Service, Analytics Service, PostgreSQL database, and Redis cache are all containerized individually. Docker Compose is used to start and manage all these containers together. This means the entire application can be launched using a single command, and all services can communicate with each other through Docker's internal network. The React frontend dashboard runs as a separate service in the local environment. It is responsible for displaying the user interface of Credit Risk Score Analytics. Through this dashboard, users can access different pages such as Dashboard, Transactions, Analytics, Risk Monitoring, API Health, and Settings. The frontend sends API requests to backend services to perform login, fetch transactions, display analytics, and check service health. The Auth Service runs as an independent backend microservice. In the local deployment setup, it commonly runs on port 8001. This service handles user registration, login, JWT token generation, password hashing, API key handling, and role-based access control. When a user logs in from the frontend, the request is sent to the Auth Service. The service verifies the user details from PostgreSQL and returns a JWT token if the credentials are valid. The Banking API Service runs as another backend microservice, commonly on port 8002. It handles payment initiation, transaction records, account-related operations, consent handling, and transaction streaming. When the frontend requests transaction history or creates a payment, the request is sent to the Banking API Service. This service communicates with PostgreSQL to store and retrieve banking data. The Analytics Service runs separately, commonly on port 8003. It is responsible for processing transaction data and generating insights such as fraud risk distribution, spending by category, transaction summary, and service health information. The Analytics Service reads transaction data from PostgreSQL and may use Redis to cache analytics responses. This improves performance because repeated analytics requests can be served faster from cache.

4.4 AWS Deployment Architecture

The AWS deployment architecture of Credit Risk Score Analytics is designed to demonstrate how a cloud-native fintech application can be deployed using modern cloud services. In this architecture, the project is moved from a local Docker-based setup to an AWS cloud environment where frontend hosting, Docker image storage, and backend service execution are handled through different AWS services. The main purpose of this deployment is to prove that Credit Risk Score Analytics is not only a locally running project but can also be hosted and executed on a real cloud platform. In the AWS deployment setup, the React frontend dashboard is hosted using Amazon S3 static website hosting. After building the React application using Vite, the generated production files are uploaded to an S3 bucket. This allows users to access the Credit Risk Score Analytics dashboard through a public S3 website endpoint. The dashboard acts as the user interface where users can view transaction data, analytics charts, fraud alerts, risk monitoring, and service health status. Hosting the frontend on S3 is suitable because it is simple, cost-effective, scalable, and reliable for static web applications. For backend deployment, the Docker images of backend services are stored in Amazon Elastic Container Registry, also known as Amazon ECR. ECR works as a private container image repository where Docker images can be pushed, stored, versioned, and pulled whenever required. In Credit Risk Score Analytics, backend services such as the Auth Service are first containerized using Docker. After building the image locally, it is tagged and pushed to Amazon ECR. This makes the backend image available for deployment on AWS container services. The Auth Service is deployed using Amazon ECS Fargate. ECS Fargate is a serverless container execution service that allows Docker containers to run in the cloud without manually managing EC2 servers. In Credit Risk Score Analytics, the Auth Service container image is pulled from Amazon ECR and executed as a task on ECS Fargate. The service exposes a public health endpoint that can be tested from a browser or terminal to confirm that the backend service is running successfully in the cloud. This proves that Credit Risk Score Analytics backend services can be deployed on cloud infrastructure using container-based deployment. The AWS deployment architecture separates the frontend and backend layers. The frontend is served from Amazon S3, while the backend service runs on ECS Fargate. When the user opens the hosted dashboard, the frontend can send API requests to the deployed backend endpoint. For example, authentication-related requests such as login, registration, and health checks are handled by the Auth Service running on ECS Fargate.

5. PROGRAMMING AND WORKING ENVIRONMENT

5.1 Frontend Technologies

The frontend of Credit Risk Score Analytics is developed using React. React is used to build the user interface of the project and helps create different dashboard pages such as Dashboard, Transactions, Analytics, Risk Monitoring, API Health, and Settings. Vite is used as the frontend build tool because it makes development faster and produces optimized production files. Recharts is used for displaying charts and graphs, while Axios is used to send API requests from the frontend to the backend services.

5.2 Backend Technologies

The backend of Credit Risk Score Analytics is developed using Python and FastAPI. Python is used as the main programming language because it is simple, powerful, and widely used for backend development, automation, data processing, and API development. FastAPI is used to build the backend microservices because it is fast, lightweight, modern, and suitable for creating REST APIs. It also provides automatic API documentation, request validation, and better performance compared to many traditional backend frameworks. Credit Risk Score Analytics backend is divided into multiple microservices such as Auth Service, Banking API Service, and Analytics Service. The Auth Service manages user registration, login, JWT token generation, password hashing, API key handling, and role-based access control. The Banking API Service handles payment initiation, transaction management, consent handling, and real-time transaction streaming. The Analytics Service processes transaction data and provides insights such as fraud risk distribution, spending by category, transaction summary, and API health information. Uvicorn is used as the ASGI server to run FastAPI applications. It helps start the backend services and makes them available through specific ports. Pydantic is used for data validation and request handling. It checks whether the data received from the frontend or API requests is valid before processing it. SQLAlchemy is used as the ORM tool to connect backend services with the PostgreSQL database and perform database operations using Python models.

5.3 Database and Caching Technologies

PostgreSQL is used as the main database of the project. It stores important information such as users, payments, transactions, account details, and consent records. Redis is used as a caching system for temporary data such as analytics results and rate-limiting information. This improves performance because frequently requested analytics data can be served quickly without repeatedly querying the database.

5.4 DevOps and Cloud Technologies

Docker is used to containerize all services. Docker Compose is used to run the complete system locally. GitHub is used for version control and GitHub Actions is included for CI/CD pipeline support. AWS S3 is used to host the frontend dashboard, Amazon ECR is used to store Docker images, and AWS ECS Fargate is used to deploy the Auth Service in the cloud without managing servers. Terraform is included for infrastructure-as-code support. Credit Risk Score Analytics uses DevOps and cloud technologies to make the project easier to build, run, test, manage, and deploy. Since the project follows a microservices architecture, DevOps tools are important for running different services together in a controlled environment. Cloud technologies are used to demonstrate that the project can move beyond local development and run on real cloud infrastructure.

Table (3): Technologies Used in Credit Risk Score Analytics

Technology	Use in Project
React	Frontend dashboard
PostgreSQL	Relational database
Redis	Caching and rate limiting
Docker Compose	Local multi-container deployment
AWS S3	Frontend static hosting
Amazon ECR	Docker image registry
AWS ECS Fargate	Cloud deployment of backend service

6.DATABASE ANALYSIS, DESIGN AND IMPLEMENTATION

6.1 Database Design

The database design of Credit Risk Score Analytics is created to store and manage all important information related to users, authentication, banking transactions, payments, accounts, consent records, and analytics data. For this purpose, PostgreSQL is used as the main relational database because it supports structured data storage, relationships between tables, data consistency, and secure transaction handling. The database design follows a relational model where different entities are stored in separate tables.

6.2 Main Database Entities

The important entities in Credit Risk Score Analytics include users, accounts, payments, transactions, consent records, API keys, and analytics-related records. User records support authentication and identity management. Payment and transaction records support banking operations. Consent records represent secure access permissions. Analytics records support reporting and visualization.

Table (4): Main Database Entities

Entity	Purpose
Users	Stores user profile and authentication information
Accounts	Represents banking accounts
Payments	Stores initiated payment records
Transactions	Stores transaction history and risk attributes
Consent Records	Stores access permissions
API Keys	Supports secure third-party access

6.3 Caching Design

The caching design of Credit Risk Score Analytics is created to improve the performance, speed, and efficiency of the system. Since Credit Risk Score Analytics deals with banking transactions, analytics data, fraud risk summaries, and dashboard information, some data may be requested again and again by the frontend. If every request is directly sent to the PostgreSQL database, it can increase database load and slow down the response time. To solve this problem, Credit Risk Score Analytics uses Redis as a caching layer between the backend services and the database. Redis is used to store temporary data that is frequently requested by the system. In Credit Risk Score Analytics, the Analytics Service uses Redis mainly for caching analytics responses. For example, data such as total transactions, fraud risk distribution, spending by category, transaction summaries, and dashboard statistics may not change every second. If the same analytics request is made repeatedly, the Analytics Service can first check Redis. If the data is already available in Redis, the service returns the cached result quickly without querying PostgreSQL again. This improves response time and reduces unnecessary database operations.

6.4 Data Processing

Data processing in Credit Risk Score Analytics is an important part of the system because the project deals with banking transactions, payments, fraud risk scoring, analytics, and dashboard visualization. The main purpose of data processing is to collect raw transaction data, validate it, store it properly, analyze it, and convert it into useful information that can be displayed on the frontend dashboard. Since banking systems generate a large amount of financial data, proper data processing helps in understanding transaction behavior, identifying suspicious activity, and improving decision-making. In Credit Risk Score Analytics, data processing starts when a user performs an action through the React frontend dashboard. For example, when a user initiates a payment, views transaction history, or opens the analytics page, the frontend sends a request to the backend services. The backend receives this request and checks whether the data is valid. Validation is important because banking applications should not accept incorrect, incomplete, or unsafe data. Pydantic is used in FastAPI to validate incoming request data before it is processed further.

7. PROGRAM STRUCTURE AND GUI CONSTRUCTION

7.1 Module Description

The Credit Risk Score Analytics project is divided into different modules to make the system organized, scalable, and easy to maintain. Since the project follows a microservices-based architecture, each module performs a specific function and communicates with other modules through APIs. The major modules of Credit Risk Score Analytics are the Auth Service Module, Banking API Service Module, Analytics Service Module, and Frontend Dashboard Module. Together, these modules provide secure authentication, banking operations, transaction processing, analytics generation, fraud monitoring, and dashboard visualization. The modular structure helps in separating responsibilities. Authentication-related tasks are handled by the Auth Service, banking operations are handled by the Banking API Service, analytics and insights are handled by the Analytics Service, and user interaction is handled by the Frontend Dashboard. This separation makes the project easier to develop, test, debug, update, and deploy.

7.2 Auth Service Module

The Auth Service Module is responsible for managing user authentication and authorization in Credit Risk Score Analytics. Since the project is related to digital banking and financial data, secure access control is one of the most important parts of the system. This module ensures that only valid and authorized users can access protected APIs and banking services. The Auth Service handles user registration, where new users can create an account by providing required details such as name, email, password, and role. Before storing the user data, the service validates the input and checks whether the email already exists. User passwords are not stored in plain text. Instead, passwords are hashed using bcrypt and then stored securely in PostgreSQL. This module also handles user login. When a user enters valid credentials, the Auth Service verifies the email and password from the database. If the credentials are correct, it generates a JWT token.

7.3 Banking API Service Module

The Banking API Service Module is responsible for handling the core banking operations of Credit Risk Score Analytics. This module acts as the main business logic layer of the system. It manages payments, transactions, account-related operations, consent records, and real-time transaction activity. One of the main functions of this module is payment initiation. When a user creates a payment request from the frontend dashboard, the request is sent to the Banking API Service. The service validates the payment details such as amount, sender account, receiver account, category, and description. After validation, the payment record is stored in PostgreSQL with its status. The module also manages transaction records. It stores and retrieves transaction data such as transaction amount, transaction type, category, date, time, country, status, and risk level. Users can view their transaction history through the frontend dashboard. The Banking API Service fetches the required transaction data from PostgreSQL and sends it back to the frontend.

7.4 Analytics Service Module

The Analytics Service Module is responsible for processing banking data and generating useful insights for the dashboard. This module reads transaction and payment data from PostgreSQL and converts it into meaningful analytics that help users understand financial activity and risk patterns. The Analytics Service generates transaction summaries such as total number of transactions, total transaction amount, recent transactions, and overall activity status. These summaries are useful for giving users a quick overview of the banking system. The module also generates fraud risk distribution. It analyzes stored transaction risk levels and shows how many transactions are low risk, medium risk, or high risk. This helps users and administrators identify whether the system has suspicious financial activity. Spending by category is another important feature of this module. The Analytics Service groups transactions based on categories such as shopping, food, travel, bills, or other spending types. This processed data is then displayed on the frontend dashboard using charts and graphs.

7.5 Frontend Dashboard Module

The Frontend Dashboard Module is responsible for providing the user interface of Credit Risk Score Analytics. It is developed using React and gives users a modern fintech-style dashboard to interact with the system. This module connects with backend services through API calls and displays the received data in a clear and visual format. The dashboard includes multiple sections such as Dashboard, Transactions, Analytics, Risk Monitoring, API Health, and Settings. The main Dashboard page displays an overview of the system, including transaction summaries, recent activity, fraud alerts, and key financial indicators. The Transactions section allows users to view transaction history. It displays transaction details such as amount, category, status, risk level, and date. This helps users monitor their banking activity in an organized way. The Analytics section displays charts and graphs generated from transaction data. It may include fraud risk distribution, spending by category, transaction trends, and summary statistics. Recharts is used to present this data visually, making it easier to understand. The Risk Monitoring section focuses on suspicious transactions and fraud-related information. It displays transactions classified as low, medium, or high risk and helps users identify unusual activities. The API Health section shows the status of backend services such as Auth Service, Banking API Service, and Analytics Service. This is useful for checking whether the system components are running correctly. The Settings section can be used to manage configuration-related options and display system details. The frontend also supports fallback demo mode so that the dashboard remains useful even if some backend APIs are unavailable. Axios is used in the frontend to send API requests to backend services. When the user logs in, views transactions, or opens analytics, Axios communicates with the required backend service and retrieves the data. The frontend then displays the data using cards, tables, charts, buttons, and status indicators. Overall, the Frontend Dashboard Module acts as the visible part of Credit Risk Score Analytics. It provides a clean, interactive, and user-friendly interface for authentication, transaction monitoring, analytics visualization, fraud risk tracking, and service health checking.

8. CODE IMPLEMENTATION AND DATABASE CONNECTIONS

8.1 Docker Implementation

The Docker implementation containerizes the frontend and backend services. The frontend uses a multi-stage Dockerfile where the React application is built using Node and then served through Nginx. This makes the frontend lightweight and production-ready. In Credit Risk Score Analytics, each major service is placed inside a separate Docker container. The frontend container runs the React dashboard, while backend containers run the FastAPI microservices. PostgreSQL and Redis also run in their own containers.

8.2 API Implementation

The API design follows REST conventions with JSON response format and structured error responses. Protected endpoints require a Bearer token in the Authorization header. The backend APIs are organized according to service responsibilities, which makes the system easier to test and maintain.

8.3 Database Connectivity

The backend services of Credit Risk Score Analytics are developed using FastAPI. These services communicate with PostgreSQL whenever data needs to be stored, updated, or fetched. For example, when a user registers, the Auth Service connects to PostgreSQL and stores the user details along with the hashed password.

8.4 Cloud Deployment Implementation

For cloud deployment proof, Docker images are pushed to Amazon ECR and the Auth Service is deployed on AWS ECS Fargate. The frontend is hosted as a static website using AWS S3. This demonstrates how a fintech application can be prepared for cloud deployment using modern DevOps practices. It demonstrates how frontend applications, backend services, and Docker images can be executed in AWS services.

9. SYSTEM TESTING

9.1 Local Testing

Local testing is performed using Docker Compose. The command `docker compose up -d` starts all microservices, frontend, PostgreSQL, and Redis in the background. After the containers are started, each service can be tested through health endpoints and API requests. The command `docker compose up -d` is used to start all containers in the background.

9.2 API Testing

API testing is performed to verify whether the backend services are working correctly. Since Credit Risk Score Analytics uses FastAPI-based microservices, each service exposes different API endpoints for specific operations. These endpoints can be tested using tools such as `curl`, browser requests, Postman, or FastAPI's automatic documentation interface. The Auth Service APIs are tested for user registration, login, JWT token generation, token verification, API key handling, and protected route access.

9.3 Frontend Testing

Frontend testing is performed to verify whether the React dashboard works properly and displays data correctly. The Credit Risk Score Analytics dashboard includes different pages such as Dashboard, Transactions, Analytics, Risk Monitoring, API Health, and Settings. Each page is tested to confirm that it loads correctly and presents information in a clear and user-friendly manner.

9.4 Cloud Testing

Cloud testing includes verifying that the frontend hosted on AWS S3 is accessible and that the Auth Service deployed on ECS Fargate responds through its public health endpoint. This proves that the project is not limited to local execution and can run in a cloud environment.

10. LIMITATIONS AND FUTURE SCOPE

10.1 Advantages

Credit Risk Score Analytics has several advantages because it combines fintech, cloud computing, cybersecurity, DevOps, microservices, and data analytics concepts in one project. It is not only a simple banking application but a cloud-ready open banking analytics platform that demonstrates how modern financial systems can be designed, secured, tested, and deployed. One major advantage of Credit Risk Score Analytics is its microservices-based architecture. The system is divided into separate services such as Auth Service, Banking API Service, and Analytics Service. Each service has its own responsibility, which makes the project easier to understand, develop, test, and maintain. If one module needs to be updated, changes can be made without disturbing the complete application. Another important advantage is secure authentication and authorization. Credit Risk Score Analytics uses JWT-based authentication, password hashing using bcrypt, API key handling, and role-based access control. These features help protect user accounts and sensitive financial data from unauthorized access. Since the project is related to banking, this security-focused design makes the system more practical and professional. Credit Risk Score Analytics also provides fraud risk monitoring. The system analyzes transactions based on factors such as amount, country, time, and unusual behavior. Based on these conditions, transactions can be categorized as low, medium, or high risk. This helps demonstrate how suspicious financial activity can be identified in a digital banking system. Another advantage is the interactive frontend dashboard. The React dashboard provides pages such as Dashboard, Transactions, Analytics, Risk Monitoring, API Health, and Settings. Users can view transaction summaries, fraud alerts, spending charts, transaction tables, and backend service status in a clear visual format. This makes the system easy to use and present during project demonstration. The project also uses PostgreSQL and Redis, which improves data handling and performance. PostgreSQL stores structured banking data such as users, payments, transactions, and consent records. Redis is used for caching analytics results and temporary data, which helps reduce database load and improves response time.

10.2 Limitations

Credit Risk Score Analytics has several advantages because it combines fintech, cloud computing, cybersecurity, DevOps, microservices, and data analytics concepts in one project. It is not only a simple banking application but a cloud-ready open banking analytics platform that demonstrates how modern financial systems can be designed, secured, tested, and deployed. One major advantage of Credit Risk Score Analytics is its microservices-based architecture. The system is divided into separate services such as Auth Service, Banking API Service, and Analytics Service. Each service has its own responsibility, which makes the project easier to understand, develop, test, and maintain. If one module needs to be updated, changes can be made without disturbing the complete application. Another important advantage is secure authentication and authorization. Credit Risk Score Analytics uses JWT-based authentication, password hashing using bcrypt, API key handling, and role-based access control. These features help protect user accounts and sensitive financial data from unauthorized access. Since the project is related to banking, this security-focused design makes the system more practical and professional. Credit Risk Score Analytics also provides fraud risk monitoring. The system analyzes transactions based on factors such as amount, country, time, and unusual behavior. Based on these conditions, transactions can be categorized as low, medium, or high risk. This helps demonstrate how suspicious financial activity can be identified in a digital banking system. Another advantage is the interactive frontend dashboard. The React dashboard provides pages such as Dashboard, Transactions, Analytics, Risk Monitoring, API Health, and Settings. This makes the system easy to use and present during project demonstration. The project also uses PostgreSQL and Redis, which improves data handling and performance. PostgreSQL stores structured banking data such as users, payments, transactions, and consent records. Redis is used for caching analytics results and temporary data, which helps reduce database load and improves response time. A major advantage of Credit Risk Score Analytics is its Docker-based implementation. Docker and Docker Compose allow all services to run together locally in separate containers. This makes the project portable and easier to run on different systems. The complete application can be started using a single Docker Compose command, which simplifies testing and demonstration. Credit Risk Score Analytics also has cloud deployment proof using AWS.

10.3 Future Scope

The future scope of Credit Risk Score Analytics is wide because the project can be improved and extended in many ways. Since the project already follows a microservices and cloud-ready architecture, new features can be added without changing the complete system structure. One major future improvement is to implement **machine learning-based fraud detection**. Instead of using only rule-based risk scoring, the system can use ML models to detect suspicious transactions more accurately. Models can be trained on transaction patterns, user behavior, transaction frequency, amount, location, device information, and historical fraud cases. This would make fraud detection more realistic and powerful. Another future enhancement is **complete AWS cloud deployment**. PostgreSQL can be moved to Amazon RDS, Redis can be moved to Amazon ElastiCache, and an Application Load Balancer can be used to route requests between services. Credit Risk Score Analytics can also be improved by adding a complete **CI/CD pipeline**. Tools such as GitHub Actions, AWS CodeBuild, AWS CodePipeline, or Jenkins can be used to automatically build Docker images, run tests, push images to Amazon ECR, and deploy services to ECS Fargate. This would make the project more professional and closer to real-world DevOps practices. Another future scope is integration with **real or sandbox banking APIs**. Open banking APIs or fintech sandbox APIs can be connected to fetch realistic account and transaction data. This would make the project more practical and useful for real banking use cases. The project can also include **multi-factor authentication** for stronger security. Features such as OTP-based login, email verification, password reset, login alerts, and session management can be added. AWS CloudWatch, Prometheus, Grafana, or ELK Stack can be used to monitor service health, API performance, error logs, container usage, and traffic patterns. Alerts can be configured to notify administrators when a service fails or when suspicious activity is detected. Another future improvement is to enhance the **frontend dashboard**. Features such as advanced filters, date range selection, export reports, transaction search, user profile settings, dark mode, notification panel, and admin dashboard can be added.

11. CONCLUSION

Credit Risk Score Analytics is a cloud-native open banking analytics platform developed to demonstrate the working of a modern digital banking and fintech system. The project successfully combines important concepts such as cloud computing, DevOps, cybersecurity, microservices architecture, database management, caching, fraud risk monitoring, and data analytics. It provides a practical understanding of how real-world banking applications manage users, payments, transactions, consent records, analytics, and security. The project is designed using a microservices-based architecture, where different services perform different responsibilities. The Auth Service handles user registration, login, JWT authentication, password hashing, API key handling, and role-based access control. The Banking API Service manages payment initiation, transaction records, consent handling, and banking-related operations. The Analytics Service processes transaction data and provides useful insights such as transaction summaries, fraud risk distribution, spending by category, and API health information. This separation of services makes the system modular, scalable, and easier to maintain. Credit Risk Score Analytics also includes a React-based frontend dashboard that provides a clear and interactive user interface. Through the dashboard, users can view transaction details, fraud alerts, analytics charts, risk monitoring information, and backend service health status. The use of visual components such as charts, cards, tables, and status indicators makes the system easy to understand and suitable for demonstration. The project also focuses strongly on security. Features such as JWT-based authentication, bcrypt password hashing, role-based access control, BOLA protection, API keys, CORS configuration, rate limiting, and security headers help protect the application from unauthorized access and common API security risks. Since the project is related to banking and financial data, these security features are very important. Docker and Docker Compose are used to containerize and run the complete system locally. This allows the frontend, backend microservices, PostgreSQL database, and Redis cache to run together in separate containers. PostgreSQL is used for permanent structured data storage, while Redis is used for caching analytics responses and improving performance.

CODE SNIPPETS AND IMPLEMENTATION FILES

```

app.py > write_dataframe_to_postgres
1  import streamlit as st
2  import pandas as pd
3  import os
4  import psycopg2
5
6  # 1. Page Config
7  st.set_page_config(page_title="FinTech Credit Analytics Engine", layout="wide")
8  st.title("FinTech Credit Risk Analytics Platform")
9
10 # DB Config from environment variables
11 DB_HOST = os.getenv("DB_HOST", "localhost")
12 DB_PORT = os.getenv("DB_PORT", "5432")
13 DB_USER = os.getenv("DB_USER", "postgres")
14 DB_PASSWORD = os.getenv("DB_PASSWORD", "password")
15 DB_NAME = os.getenv("DB_NAME", "fintech_analytics")
16
17
18 def write_dataframe_to_postgres(df: pd.DataFrame):
19     conn = psycopg2.connect(
20         host=DB_HOST,
21         port=DB_PORT,
22         database=DB_NAME,
23         user=DB_USER,
24         password=DB_PASSWORD
25     )
26
27     cursor = conn.cursor()
28
29     cursor.execute("""
30         CREATE TABLE IF NOT EXISTS credit_analytics (
31             customer_id INTEGER,
32             income NUMERIC,
33             monthly_debt NUMERIC,
34             credit_score INTEGER,
35             employment_years INTEGER,
36             dti DOUBLE PRECISION,
37             status TEXT,
38             interest_rate DOUBLE PRECISION

```

Figure (3): App.py for Credit Risk Score Analytics Frontend


```

83
84
85 # 3. Main Body Pipeline
86 if 'batch_df' in st.session_state:
87     st.subheader("📄 Ingested Raw Financial Records")
88     st.dataframe(st.session_state['batch_df'], use_container_width=True)
89
90     if st.button("🚀 Run Analytics Pipeline"):
91         with st.spinner("Running analytics transformations..."):
92             processed_pdf = st.session_state['batch_df'].copy()
93             processed_pdf['dti'] = processed_pdf['monthly_debt'] / (processed_pdf['income'] / 12)
94             processed_pdf['status'] = processed_pdf.apply(
95                 lambda row: "Rejected" if (row['credit_score'] < 600 or row['dti'] > 0.45) else "Approved",
96                 axis=1
97             )
98             processed_pdf['interest_rate'] = 0.0
99             approved_mask = processed_pdf['status'] == "Approved"
100             processed_pdf.loc[approved_mask, 'interest_rate'] = 0.05 + (0.10 * processed_pdf.loc[approved_mask, 'dti'])
101
102             st.success("✅ Analytics pipeline completed successfully with pandas.")
103
104             col1, col2 = st.columns(2)
105             with col1:
106                 st.metric("Total Records Processed", len(processed_pdf))
107                 st.subheader("Processed Results")
108                 st.dataframe(processed_pdf, use_container_width=True)
109
110             with col2:
111                 st.subheader("Risk Status Distribution")
112                 st.bar_chart(processed_pdf['status'].value_counts())
113
114             try:
115                 write_dataframe_to_postgres(processed_pdf)
116                 st.info("📄 Analytics data successfully logged to AWS RDS.")
117             except Exception as e:
118                 st.warning("⚠️ Database write skipped or failed: {e}")
119

```

Figure (4): Main Body Pipeline

```

postgres=# \l
postgres=#
postgres=# docker inspect fintech-db | grep POSTGRES
postgres=# docker inspect fintech-db
postgres=# cd
postgres=# \q
ubuntu@ip-172-31-21-87:~$ docker exec fintech-db env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/bin:/sbin:/bin:/usr/lib/postgresql/15/bin
HOSTNAME=1b5ealb4414a
POSTGRES_DB=fintech_analytics
POSTGRES_USER=postgres
POSTGRES_PASSWORD=admin123
GOSU_VERSION=1.19
LANG=en_US.utf8
PG_MAJOR=15
PG_VERSION=15.18-1.pgdg13+1
PGDATA=/var/lib/postgresql/data
HOME=/root
ubuntu@ip-172-31-21-87:~$
l.000f11d7df6d848fa (streamlit.fintech-server)

```

Figure (5): Ubuntu Docker Setup

TESTING OUTPUTS AND DEPLOYMENT PROOF



Figure (6): WebPage

The screenshot shows the 'Ingested Raw Financial Records' section of the platform. It features a table with financial data and a 'Run Analytics Pipeline' button.

	customer_id	income	monthly_debt	credit_score	employment_years
0	101	45,000	2,000	550	1
1	102	120,000	1,500	740	5
2	103	30,000	1,800	620	2
3	104	85,000	2,500	680	4
4	105	60,000	4,000	560	0

Below the table is a button labeled 'Run Analytics Pipeline'.

Figure (7): Audit Records

Table (5): Testing Summary

Test Area	Expected Result	Status
Docker Compose local deployment	All containers start successfully	Passed
Frontend dashboard	Dashboard loads and displays data	Passed
AWS deployment proof	Auth Service responds from ECS Fargate public endpoint	Passed

REFERENCES

1. Docker Inc., *Docker Documentation*, Available at: <https://docs.docker.com/>
2. Amazon Web Services (AWS), *Amazon EC2 Documentation*, Available at: <https://docs.aws.amazon.com/ec2/>
3. PostgreSQL Global Development Group, *PostgreSQL Official Documentation*, Available at: <https://www.postgresql.org/docs/>
4. Streamlit Inc., *Streamlit Documentation*, Available at: <https://docs.streamlit.io/>
5. Python Software Foundation, *Python Documentation*, Available at: <https://docs.python.org/3/>
6. Pandas Development Team, *Pandas User Guide*, Available at: <https://pandas.pydata.org/docs/>
7. IBM Cloud Education, “What is DevOps?”, Available at: <https://www.ibm.com/topics/devops>
8. Amazon Web Services, *AWS Cloud Computing Concepts*, Available at: <https://aws.amazon.com/what-is-cloud-computing/>
9. Red Hat, *Introduction to Containerization and Docker*, Available at: <https://www.redhat.com/>
10. Oracle, *Introduction to Database Management Systems*, Available at: <https://www.oracle.com/database/>
11. Financial Technology Association, *Introduction to Financial Technology (FinTech)*, Available at: <https://www.ftassociation.org/>
12. Ubuntu Documentation, *Ubuntu Server Guide*, Available at: <https://ubuntu.com/server/docs>
13. GitHub Documentation, *Version Control and Repository Management*, Available at: <https://docs.github.com/>
14. Kurose, J. F., & Ross, K. W., *Computer Networking: A Top-Down Approach*, Pearson Education.
15. Pressman, R. S., *Software Engineering: A Practitioner’s Approach*, McGraw-Hill Education.

APPENDIX

Appendix A: Main Local Deployment Command

The main command used for local deployment is given below:

```
docker compose up -d
```

After running this command, all services start in the background. The frontend dashboard becomes available at <http://localhost:3000>, the Auth Service runs at <http://localhost:8001>, the Banking API Service runs at <http://localhost:8002>, and the Analytics Service runs at <http://localhost:8003>.

Appendix B: Cloud Deployment Summary

The frontend dashboard is hosted on AWS S3, backend Docker images are stored in Amazon ECR, and the Auth Service is deployed on AWS ECS Fargate. This setup provides cloud deployment proof for the project and can be extended by deploying all backend services with RDS, ElastiCache, and an Application Load Balancer.

